

Theming et conception sous WordPress



Qu'est ce que WordPress ?

WordPress est un CMS créé en 2003, permettant de créer rapidement des sites web.

Presque 1 site sur 2 l'utilise, 43% des sites tournent sous WordPress

(React, Vue et Angular combinés dépassent à peine 1%).

WordPress est écrit en PHP et s'appuie sur une base de données MySQL.

WordPress.com vs WordPress.org

WordPress.com :

- Service d'hébergement "clé en main"
- Payant et limité en fonction de son abonnement
- Idéal pour débiter rapidement ou pour les gens sans expérience

WordPress.org :

- Logiciel gratuit et open-source
- Permet une personnalisation totale

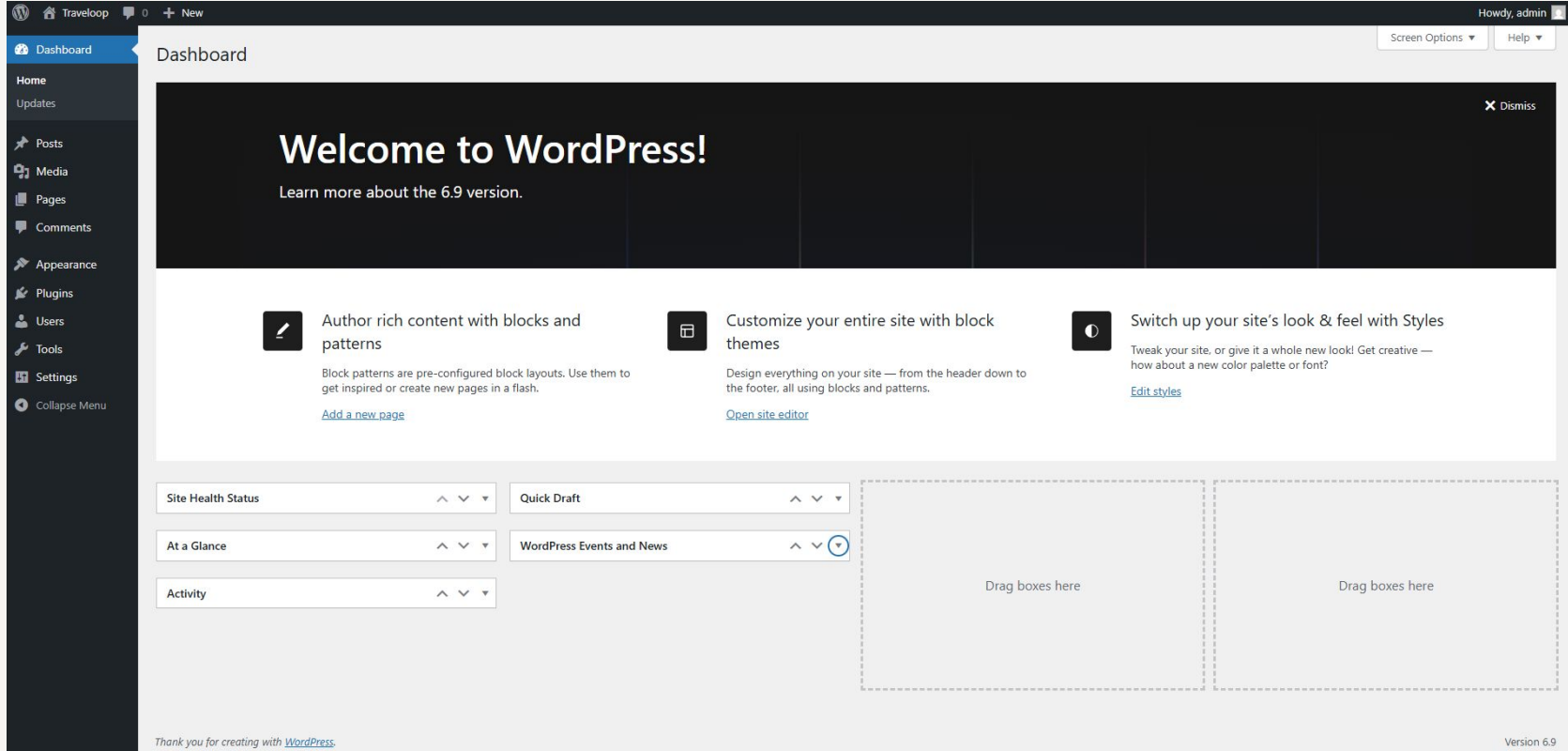
Local by flywheel

The screenshot displays the Local by Flywheel application window. On the left is a green sidebar with icons for user profile, local sites, cloud sync, file explorer, settings, help, and a plus sign for adding new sites. The main panel shows the configuration for a site named 'Travelloop'. At the top right of the main panel, there is a 'Stop site' button and a clock icon indicating 'Last started: Today'. Below the site name are links for 'Go to site folder', 'Site shell', and 'VS Code'. A tabbed interface shows 'Overview' (selected), 'Database', and 'Tools'. Two buttons, 'WP Admin' and 'Open site', are located to the right of the tabs. The configuration table lists various settings:

Site domain	travelloop.local	Change
SSL	travelloop.local.crt	Trust i
Web server	nginx	
PHP version	8.1.29	Details ↗
Database	MySQL 8.0.16	
One-click admin	☒ Off	Select admin i
WordPress version	6.9	
Multisite	No	
Xdebug	☒ Off	Details ↗

At the bottom of the sidebar, it shows '1 site running' and a 'Stop all' button. The bottom of the main panel features a 'Live Link' toggle (currently disabled) and 'Pull' and 'Push' buttons with cloud icons.

La base de WordPress



The screenshot shows the WordPress 6.9 Dashboard. At the top, there's a header bar with the WordPress logo, the site name 'Traveloop', a notification icon, a '+ New' button, and the user name 'Howdy, admin' with a profile icon. Below the header, the left sidebar contains a 'Dashboard' button and a 'Home' section with 'Updates' and a list of menu items: Posts, Media, Pages, Comments, Appearance, Plugins, Users, Tools, Settings, and Collapse Menu. The main content area has a 'Dashboard' title and a 'Screen Options' button. A large dark banner at the top of the main area says 'Welcome to WordPress!' and 'Learn more about the 6.9 version.' Below this, there are three cards: 'Author rich content with blocks and patterns' with a link to 'Add a new page', 'Customize your entire site with block themes' with a link to 'Open site editor', and 'Switch up your site's look & feel with Styles' with a link to 'Edit styles'. At the bottom, there's a grid of widgets: 'Site Health Status', 'Quick Draft', 'At a Glance', 'WordPress Events and News', and 'Activity'. To the right of these widgets are two large dashed boxes labeled 'Drag boxes here'. At the very bottom, there's a footer with 'Thank you for creating with WordPress' and 'Version 6.9'.

WordPress 6.9 Dashboard

Howdy, admin

Dashboard

Welcome to WordPress!

Learn more about the 6.9 version.

Author rich content with blocks and patterns

Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.

[Add a new page](#)

Customize your entire site with block themes

Design everything on your site — from the header down to the footer, all using blocks and patterns.

[Open site editor](#)

Switch up your site's look & feel with Styles

Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?

[Edit styles](#)

Site Health Status

Quick Draft

At a Glance

WordPress Events and News

Activity

Drag boxes here

Drag boxes here

Thank you for creating with [WordPress](#)

Version 6.9

Comment fonctionne WordPress

WordPress a deux principales entrées de contenu :

- les pages : permettant de créer un contenu statique, n'ayant pas vocation à évoluer (page de contact, landing page).
- les posts : permettant de créer du contenu évolutif (actualités, événements).

Les médias dans WordPress

Tous les fichiers (images, PDF, vidéos...) sont gérés via la Médiathèque.

Chaque média peut être :

- attaché à un contenu
- réutilisé librement dans le site

Les médias sont des contenus stockés en base (au même titre que les pages/articles).

Le rendu visuel est contrôlé par le thème (CSS + templates).

Le rôle exact d'un thème WordPress

Un thème WordPress est responsable uniquement de l'affichage du contenu.

Il ne crée pas de donnée, il ne définit pas la logique métier, il ne doit pas contenir de fonctionnalités critiques.

Son rôle est de transformer des données WordPress (posts, pages, ...) en HTML, CSS et JavaScript.

Thème classique vs Thème de blocs

Aujourd'hui, il existe deux principales approches pour créer un thème WordPress.

Les thèmes classiques (Classic Themes)

Le thème est construit avec des templates PHP (single.php, page.php, header.php...), et l'éditeur de blocs sert principalement à construire le contenu des pages et articles.

Les thèmes de blocs (Block Themes / Full Site Editing)

Le thème est construit autour des blocs, de templates HTML, et d'un fichier theme.json. On peut éditer le contenu mais aussi la structure du site (header, footer, templates) directement dans l'éditeur.

Thème existant, enfant et custom

Un **thème existant** est un thème fourni par WordPress ou par n'importe quelle personne. Il peut-être payant et est souvent fait pour répondre aux besoins de sites génériques.

Un **thème enfant** est un thème dérivé d'un thème existant (ici appelé parent), que l'on va étendre avec de nouvelles fonctionnalités.

Un **thème custom** est créé de zéro, souvent pour un besoin bien spécifique.

Theming WordPress classique

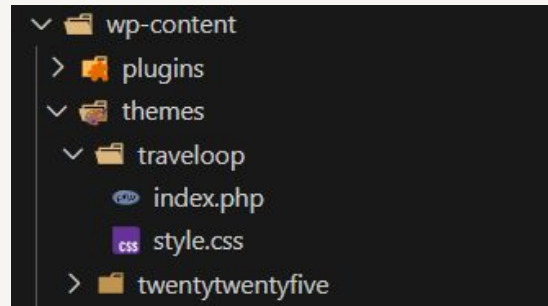


Déclarer un nouveau thème

Un thème WordPress minimal fonctionne avec :

- style.css → *déclaration du thème*
- index.php → *template de secours*
- screenshot.png (optionnel) → *image du thème*

Tous les autres fichiers sont **optionnels**, mais permettent une meilleure organisation et lisibilité du code.



Déclarer un nouveau thème

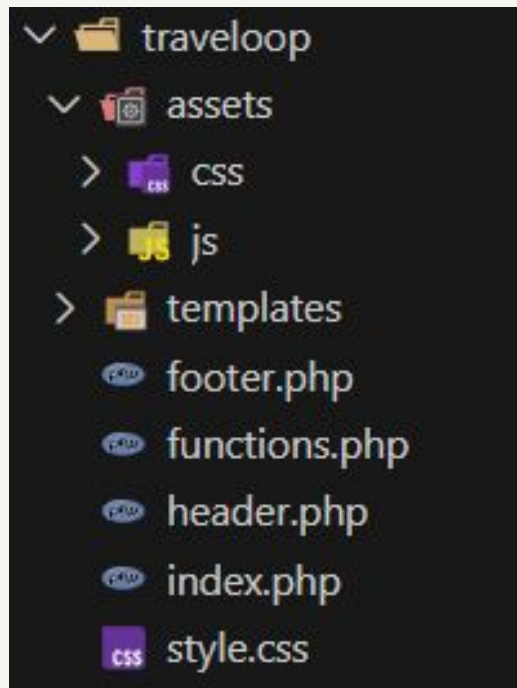
Dans le fichier style.css à la racine du projet

```
1  /*
2  Theme Name: Traveloop
3  Theme URI: https://traveloop.com
4  Author: Nicolas Vero
5  Description: WordPress theme for a travel agency
6  Version: 1.0
7  */
```

Structure d'un thème

Pour maintenir un thème propre et évolutif, on adopte une structure claire :

- séparation du PHP et des assets
- découpage des templates
- centralisation des fonctionnalités



Le fichier functions.php

Le fichier functions.php est chargé automatiquement par WordPress.

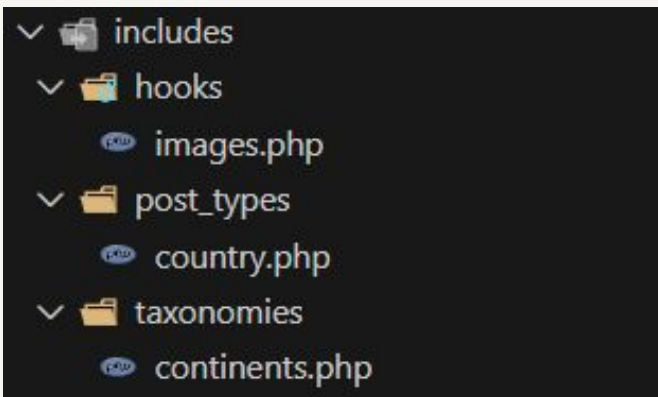
Il permet de déclarer l'ensemble des fonctions qui seront utilisées par notre thème (utilitaire, hooks, custom post type).

Il **ne doit jamais contenir de HTML** (séparation de la logique et du rendu).

Le fichier functions.php

Le fichier functions.php est l'unique point d'entrée, mais il est possible de le découper en sous fichiers inclus dans functions.php.

Le but est d'éviter qu'il devienne un "fichier poubelle" de plusieurs milliers de lignes.

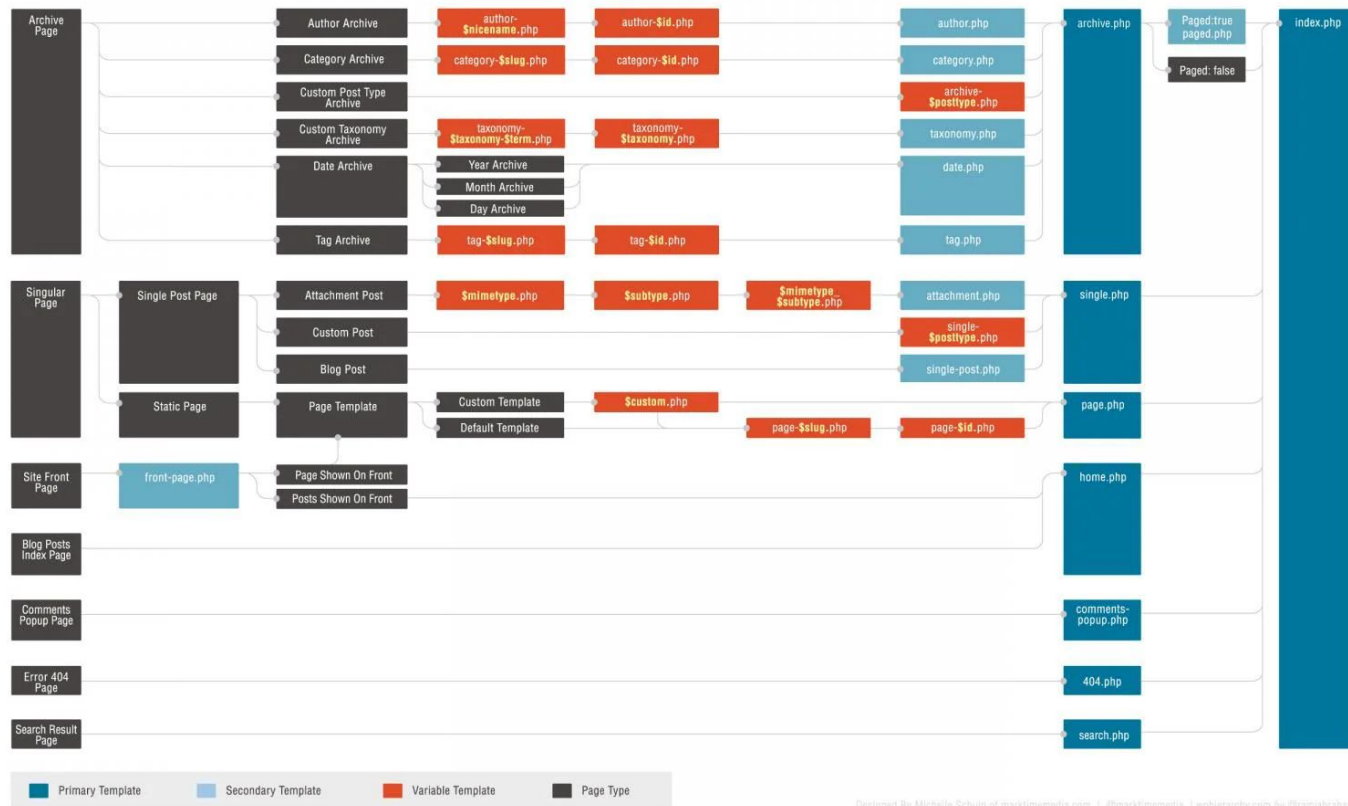


```
1 require_once get_template_directory() . '/includes/hooks/images.php';  
2 require_once get_template_directory() . '/includes/post_types/country.php';  
3 require_once get_template_directory() . '/includes/taxonomies/continents.php';
```


Comment WordPress choisit un template

Lorsqu'un utilisateur visite une page, WordPress :

1. Analyse l'URL demandée
2. Identifie le type de contenu
3. Parcourt la template hierarchy
4. Utilise le premier fichier correspondant trouvé



Designed By Michelle Schulp of marktimedia.com | @marktimedia | wphierarchy.com by @ramiabraham

Templates de pages

Pour une page de contact :

- page-**contact**.php - *slug*
- page-**12**.php - *id*
- page.php
- singular.php
- index.php

Templates de posts

Pour un article (single) :

- single-**premier-voyage**.php - *slug*
- single-**42**.php - *id*
- single.php
- singular.php
- index.php

Header et footer

WordPress permet de charger sur ses pages le contenu de notre header et de notre footer.

Pour cela, il nous fournit deux fonctions :

- `get_header()` → charge le fichier `header.php`
- `get_footer()` → charge le fichier `footer.php`

Il est également possible de charger des éléments custom :

- `get_header('custom')` charge le fichier `'header-custom.php'`.
- `get_footer('custom')` charge le fichier `'footer-custom.php'`.

Header et footer

```
1 <!DOCTYPE html>
2 <html <?php language_attributes(); ?>>
3 <head>
4     <meta charset="<?php bloginfo('charset'); ?>">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title><?php wp_title(); ?></title>
7
8     <?php wp_head(); ?>
9 </head>
10 <body <?php body_class(); ?>>
```

```
1 <?php wp_footer(); ?>
2 </body>
3 </html>
```

```
1 <?php
2
3 get_header();
4
5 // Contenu de la page
6
7 get_footer();
```

Ajouter du style et des scripts à nos pages

WordPress fournit des fonctions pour nous permettre d'injecter proprement des scripts et des fichiers de style.

Ils peuvent être définis dans `functions.php` (*conseillé*) ou dans des templates.

```
1 wp_enqueue_style('country-style', get_stylesheet_directory_uri() . '/dist/css/pages/countries.css');
2 wp_enqueue_script('country-script', get_stylesheet_directory_uri() . '/js/pages/countries.js', array(), '1.0', true);
```

```
1 function traveloop_assets() {
2     // Chargement du CSS principal
3     wp_enqueue_style('main-style', get_stylesheet_uri(), [], '1.0');
4
5     // Chargement d'un script JS avec dépendance
6     wp_enqueue_script('main-js', get_template_directory_uri() . '/js/app.js', [], '1.0', true);
7 }
8 add_action('wp_enqueue_scripts', 'mon_theme_assets');
```

Custom Post Types

La fonction `register_post_type()` permet de définir des types personnalisés en plus de “Post”.

```
1  register_post_type(  
2      string $post_type,  
3      array|string $args = array()  
4  ): WP_Post_Type|WP_Error
```

https://developer.wordpress.org/reference/functions/register_post_type/

Custom Post Types

```
1 function travel_register_country_cpt_complete_simple() {
2     $labels = array(
3         'name'           => 'Countries',
4         'singular_name'  => 'Country',
5         'menu_name'      => 'Countries',
6         'all_items'      => 'All Countries',
7         'add_new'        => 'Add New',
8         'add_new_item'   => 'Add New Country',
9         'edit_item'      => 'Edit Country',
10        'new_item'       => 'New Country',
11        'view_item'      => 'View Country',
12        'search_items'   => 'Search Countries',
13        'not_found'      => 'No countries found.',
14        'not_found_in_trash' => 'No countries found in Trash.',
15        'featured_image' => 'Country Flag or Cover Image',
16        'set_featured_image' => 'Set cover image',
17        'remove_featured_image' => 'Remove cover image',
18    );
19
20    $args = array(
21        'labels'           => $labels,
22        'description'     => 'Travel destinations by country.',
23        'public'          => true,
24        'has_archive'     => true,
25        'show_in_rest'    => true,
26        'menu_icon'       => 'dashicons-location-alt',
27        'supports'        => [ 'title', 'editor', 'thumbnail', 'excerpt' ],
28        'rewrite'         => [ 'slug' => 'country' ],
29    );
30
31    register_post_type('country', $args);
32 }
33
34 add_action('init', 'travel_register_country_cpt_complete_simple');
```

L'attribut 'menu_icon' permet de définir l'icône qui sera utilisée dans l'admin de WordPress

<https://developer.wordpress.org/resource/dashicons/>

Le *add_action()* est primordial : il permet à WordPress de prendre une fonction et de l'exécuter. Le premier paramètre de la fonction définit le moment de l'exécution, ici à **l'initialisation**.

Custom Post Types

The screenshot shows the WordPress 6.9 dashboard. At the top, the navigation bar includes the WordPress logo, the site name 'Traveloop', a notification icon with '0', a '+ New' button, and the user profile 'Howdy, admin' with a dropdown menu containing 'Screen Options' and 'Help'. The left sidebar lists menu items: Dashboard (active), Home, Updates, Posts, Media, Pages, Comments, Countries, Appearance, Plugins, Users, Tools, Settings, and Collapse Menu. The main content area features a large 'Welcome to WordPress!' banner with a 'Dismiss' button and a link to 'Learn more about the 6.9 version.'. Below the banner are three cards: 'Author rich content with blocks and patterns' (with a pencil icon), 'Start Customizing' (with a square icon), and 'Discover a new way to build your site.' (with a circle icon). Each card includes a brief description and a link. The bottom section contains a grid of widgets: 'Site Health Status', 'Quick Draft', 'At a Glance', 'WordPress Events and News', and 'Activity'. To the right of these widgets are two large dashed boxes labeled 'Drag boxes here'. At the bottom left, a footer message says 'Thank you for creating with WordPress.', and at the bottom right, it says 'Version 6.9'.

WordPress 6.9 Dashboard

Howdy, admin

Dashboard

Welcome to WordPress!

[Learn more about the 6.9 version.](#)

Author rich content with blocks and patterns

Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.

[Add a new page](#)

Start Customizing

Configure your site's logo, header, menus, and more in the Customizer.

[Open the Customizer](#)

Discover a new way to build your site.

There is a new kind of WordPress theme, called a block theme, that lets you build the site you've always wanted — with blocks and styles.

[Learn about block themes](#)

Site Health Status

Quick Draft

At a Glance

WordPress Events and News

Activity

Drag boxes here

Drag boxes here

Thank you for creating with [WordPress](#).

Version 6.9

Régénérer les permaliens

`/wp-admin/options-permalink.php`

Pour que nos custom post types soient bien pris en compte par WordPress, il est important de régénérer les permaliens.

Ceux-ci définissent les routes vers nos pages.

Cela permet de vider le cache que WordPress crée pour les routes pour des raisons de performances.

Templates de custom posts

Pour un custom post type (single) :

- single-**country**-**premier-voyage**.php - *slug*
- single-**country**-**42**.php - *id*
- single-**country**.php
- single.php
- singular.php
- index.php

Templates de custom posts

Pages	Post	Custom Post Type
page-{slug}.php	single-{slug}.php	single-{type}-{slug}.php
page-{id}.php	single-{id}.php	single-{type}-{id}.php
		single-{type}.php
page.php	single.php	single.php
singular.php	singular.php	singular.php
index.php	index.php	index.php

Les archives de posts

Les archives sont un template permettant de ressortir l'ensemble des éléments d'un même post type sur une page. Ils sont créés via les templates :

- archive-**country**.php
- archive.php

Si on a créé 3 posts de type country, l'archive nous donnera les trois.

/country - archive des Custom Post Types pays

/country/france - un Custom Post Type pays avec le slug 'france'

Afficher du contenu : The Loop

La Loop est la boucle de WordPress qui affiche les contenus.

Elle parcourt les résultats de la page en cours et affiche chaque post successivement (titre, contenu, image, etc.).

```
1  if (have_posts()) {  
2      while (have_posts()) {  
3          the_post();  
4          the_title();  
5          the_content();  
6      }  
7  }
```

Les plugins WordPress

Un plugin WordPress permet d'ajouter ou de modifier des fonctionnalités, sans dépendre du thème utilisé.

Un plugin peut :

- ajouter une logique métier
- créer ou modifier des données
- étendre le comportement de WordPress

Contrairement au thème, un plugin ne gère pas l'affichage du site.

Une fonctionnalité essentielle doit toujours être placée dans un plugin, afin qu'elle reste active même si le thème change.



Les plugins WordPress : Classic Editor

Le Classic Editor est un plugin permettant d'utiliser l'éditeur historique de WordPress. Gutenberg reste le standard WordPress, mais dans les projets "fields-first" on utilise souvent Classic Editor.

Dans de nombreux projets utilisant des champs personnalisés (SCF), l'édition du contenu se fait principalement via ces champs.

Dans ce contexte, Gutenberg est souvent peu utilisé, et le Classic Editor permet de :

- simplifier l'interface d'édition
- éviter la redondance entre champs et blocs
- se concentrer sur la structure des données



Les plugins WordPress : Classic Editor

Privacy Policy

Who we are

Suggested text: Our website address is: <http://traveloop.local>.

Comments

Suggested text: When visitors leave comments on the site we collect the data shown in the comments form, and also the visitor's IP address and browser user agent string to help spam detection.

An anonymized string created from your email address (also called a hash) may be provided to the Gravatar service to see if you are using it. The Gravatar service privacy policy is available here: <https://automattic.com/privacy/>. After approval of your comment, your profile picture is visible to the public in the context of your comment.

Media

Suggested text: If you upload images to the website, you should avoid uploading images with embedded location data (EXIF GPS) included. Visitors to the website can download and extract any location data from images on the website.

Page Metadata Sidebar:

- Page: Privacy Policy
- 610 words, 3 minutes read time. Last edited 28 minutes ago.
- Status: Draft
- Publish: Immediately
- Slug: privacy-policy
- Author: admin
- Discussion: Pings only
- Parent: None
- Move to trash



Les plugins WordPress : Classic Editor

The screenshot shows the WordPress Classic Editor interface. On the left is a dark sidebar with navigation links: Dashboard, Posts, Media, Pages (highlighted), All Pages, Add Page, Comments, Countries, Appearance, Plugins, Users, Tools, Settings, and Collapse Menu. The main content area is titled 'Edit Page' with an 'Add Page' button. Below this is the title 'Privacy Policy' and a permalink: <http://traveloop.local/privacy-policy/> with an 'Edit' button. A rich text editor toolbar is visible with options like Paragraph, Bold, Italic, Bulleted List, Numbered List, Quote, Link, and Image. The page content includes sections for 'Who we are', 'Comments', 'Media', and 'Cookies', each with a 'Suggested text' placeholder. The right sidebar contains 'Publish' controls (Save Draft, Preview, Status: Draft, Visibility: Public, Publish on: Dec 12, 2025 at 23:31) and 'Page Attributes' (Parent: (no parent), Order: 0).

Howdy, admin

Screen Options Help

Edit Page [Add Page](#)

Privacy Policy

Permalink: <http://traveloop.local/privacy-policy/> [Edit](#)

[Add Media](#) Visual Code

Paragraph B I

Who we are

Suggested text: Our website address is: <http://traveloop.local/>

Comments

Suggested text: When visitors leave comments on the site we collect the data shown in the comments form, and also the visitor's IP address and browser user agent string to help spam detection.

An anonymized string created from your email address (also called a hash) may be provided to the Gravatar service to see if you are using it. The Gravatar service privacy policy is available here: <https://automattic.com/privacy/>. After approval of your comment, your profile picture is visible to the public in the context of your comment.

Media

Suggested text: If you upload images to the website, you should avoid uploading images with embedded location data (EXIF GPS) included. Visitors to the website can download and extract any location data from images on the website.

Cookies

Suggested text: If you leave a comment on our site you may opt-in to saving your name, email address and website in cookies. These are for your convenience so that you do not have to fill in your details again when you leave another comment. These cookies will last for one year.

If you visit our login page, we will set a temporary cookie to determine if your browser accepts cookies. This cookie contains no personal data and is discarded when you close your browser.

When you log in, we will also set up several cookies to save your login information and your screen display choices. Login cookies last for two days, and screen options cookies last for a year. If you select "Remember Me", your login will persist for two weeks. If you log out of your account, the login cookies will be removed.

If you edit or publish an article, an additional cookie will be saved in your browser. This cookie includes no personal data and simply indicates the post ID of the article you just edited. It serves

Publish

[Save Draft](#) [Preview](#)

Status: **Draft** [Edit](#)

Visibility: **Public** [Edit](#)

Publish on: Dec 12, 2025 at 23:31 [Edit](#)

[Move to Trash](#) [Publish](#)

Page Attributes

Parent

(no parent)

Order

0

Need help? Use the Help tab above the screen title.

Les plugins WordPress : SCF

Secure Custom Fields (SCF) permet de créer des champs personnalisés structurés, associés à un contenu WordPress (page, post, CPT).

Ces champs permettent de :

- séparer le contenu de la mise en forme
- guider les intégrateurs dans la saisie
- éviter le contenu en dur dans les templates

Les valeurs saisies sont stockées en base de données et peuvent être utilisées dans les templates PHP du thème.

<https://fr.wordpress.org/plugins/secure-custom-fields/>

Les plugins WordPress : SCF

Select Field Type

[Popular](#) [Basic](#) [Content](#) [Choice](#) [Relational](#) [Advanced](#) [Layout](#) [PRO](#)

T
Text

T
Text Area

@
Email

🌐
URL

📎
File

☑
Select

👁
True / False

🔗
Link

📄
Post Object

🔗
Relationship

∞
Repeater

🖼
Flexible Content

🖼
Gallery

📄
Clone

Cancel

Select Field

Text

A basic text input, useful for storing single string values.

Movie title

Les plugins WordPress : SCF

 Fields 

#	Label	Name	Type
1	Country name	country_name	 Text
2	Country image	country_image	 Image
3	Country continent	country_continent	 Radio Button

+ Add Field

 Settings 

Location Rules Presentation Group Settings

Rules 

Show this field group if

Post Type

is equal to

Country

and

or

Add rule group

Les plugins WordPress : SCF

Country ^ v ▲

Country name

Country image
No image selected [Add Image](#)

Country continent
☒ Africa
☐ Antarctica
☐ Asia
☐ Europe
☐ North America
☐ Oceania
☐ South America

La récupération de champs SCF

```
1  $post_id = get_the_ID();  
2  
3  $country_name = get_field('country_name');  
4  $country_image = get_field('country_image');  
5  $country_continent = get_field('country_continent');
```


Le champ répéteur

4

Country highlights

country_highlights

Repeater

General

Validation

Presentation

Conditional Logic

Field Type

Repeater

Browse Fields

Field Label

Country highlights

This is the name which will appear on the EDIT page

Field Name

country_highlights

Single word, no spaces. Underscores and dashes allowed

Sub Fields

Fields

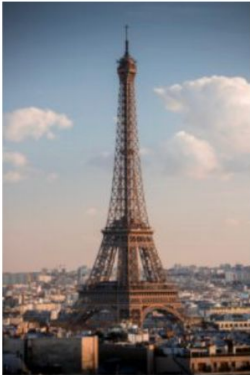
+ Add Field

#	Label	Name	Type
1	Highlight name	highlight_name	Text
2	Highlight image	highlight_image	Image

+ Add Field

Le champ répéteur

Country highlights

	Highlight name	Highlight image	
1	Tour eiffel		
2	Château de Chambord		

Add Row

Le champ répéteur

```
1  if (have_rows('country_highlights')):  
2      while (have_rows('country_highlights')): the_row();  
3          $name = get_sub_field('highlight_name');  
4          $img  = get_sub_field('highlight_image');  
5      endwhile;  
6  endif;
```

Le template single-country.php

```
1  <?php
2  $post_id = get_the_ID();
3
4  $country_name = get_field('country_name');
5  $country_image = get_field('country_image');
6  $country_continent = get_field('country_continent');
7  ?>
8
9  <section>
10     
11     <p><?= "$country_name $country_continent" ?></p>
12 </section>
```

Le template archive-country.php

```
1  <?php
2  get_header();
3  ?>
4
5  <main class="country-archive">
6      <h1>All Countries</h1>
7
8      <?php if (have_posts()): ?>
9          <div class="country-archive-list">
10              <?php while (have_posts()): the_post();
11                  $country_name = get_field('country_name');
12                  $country_image = get_field('country_image');
13                  $country_continent = get_field('country_continent');
14              ?>
15              <article class="country-card-component">
16                  <?php if ($country_image): ?>
17                      " />
18                  <?php endif; ?>
19                  <p><?=$country_name $country_continent ?></p>
20                  <a href="<?=$get_permalink() ?>">Read more</a>
21              </article>
22          <?php endwhile; ?>
23      </div>
24  <?php endif; ?>
25 </main>
26
27 <?php get_footer(); ?>
```

Les templates parts

WordPress possède un système permettant d'aller chercher des templates définis.

Cela se rapproche du système de composants que l'on peut avoir en React.

```
1 get_template_part('templates/country/card');
```

Taxonomy

Une taxonomie sous WordPress est une façon de regrouper et de classer du contenu similaire ou lié.

C'est le mécanisme qui permet d'organiser des types de contenu pour les rendre plus faciles à naviguer et à trouver.

WordPress propose deux taxonomies de base :

- catégories : hiérarchique - Actualités > International
- étiquettes : non-hiérarchiques (mots-clés simples) - #Urgent

Custom Taxonomies

WordPress permet de définir ses propres taxonomies via la fonction `register_taxonomy()` dans `functions.php`. Les taxonomies sont associées à un ou plusieurs Post Types.

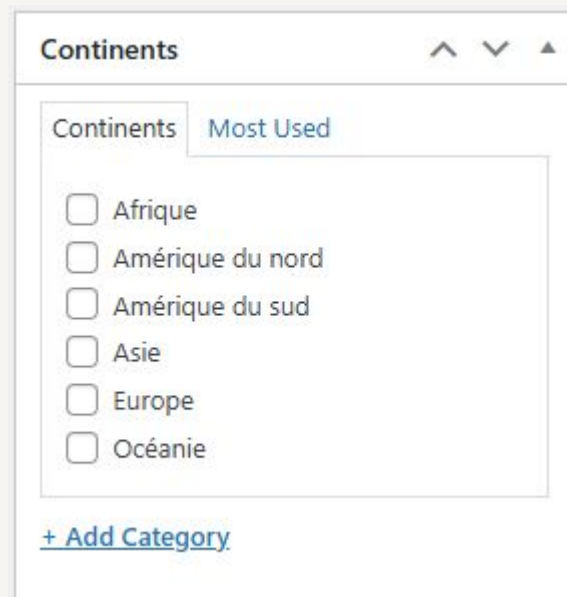
Par exemple, pour notre site de voyage, nous pourrions avoir une taxonomie avec les différents continents.

Avec cette nouvelle taxonomie, nous pourrions par exemple :

- afficher une liste de pays par continent.
- créer des filtres de recherche précis.
- générer des URLs propres (ex: `/continent/europe/`).

Custom Taxonomies

```
1 function traveloop_register_continent_taxonomy() {  
2     register_taxonomy(  
3         'continent',  
4         ['country'],  
5         [  
6             'labels' => [  
7                 'name'          => 'Continents',  
8                 'singular_name' => 'Continent',  
9             ],  
10            'public'          => true,  
11            'hierarchical'    => true,  
12            'show_admin_column' => true,  
13            'rewrite'         => [  
14                'slug' => 'continent',  
15            ],  
16        ]  
17    );  
18 }  
19 add_action('init', 'traveloop_register_continent_taxonomy');
```



Les requêtes en base avec WP_Query

```
1 $query = new WP_Query([
2     'post_type' => 'country',
3     'post_status' => 'publish',
4     'posts_per_page' => 12,
5     'tax_query' => [
6         [
7             'taxonomy' => 'continent',
8             'field' => 'slug',
9             'terms' => 'europe',
10        ],
11    ],
12    'meta_query' => [
13        [
14            'key' => 'population',
15            'value' => 10000000,
16            'type' => 'NUMERIC',
17            'compare' => '>=',
18        ],
19    ]
20 ]);
```

WP_Query est une classe permettant d'aller faire des requêtes directement dans la base de donnée.

WP_Query crée une requête secondaire en plus de la principale que WordPress génère pour la page.

Après une requête secondaire, il est nécessaire d'appeler la fonction `wp_reset_postdata()` pour restaurer le contexte global (WordPress retourne à l'état d'avant notre requête).

Les requêtes en base avec WP_Query

```
1  $query = new WP_Query([
2      'post_type' => 'country',
3      'posts_per_page' => 3,
4  ]);
5
6  if ($query->have_posts()) {
7      ?><div class="country-card-list"><?php
8      while ($query->have_posts()) {
9          $query->the_post();
10         get_template_part('templates/country/card');
11     }
12     wp_reset_postdata();
13     ?></div><?php
14 }
```

Hook WordPress

Les Hooks sont des fonctions qui permettent d'interagir avec le coeur de WordPress à des moments clés d'exécution.

Il existe deux types de hooks :

- les **actions**, permettent d'exécuter du code à un moment précis
- les **filtres**, permettent de modifier une valeur avant qu'elle ne soit utilisée

Par exemple, on pourrait :

- calculer le nombre de mots d'un article à chaque mise à jour pour estimer le temps de lecture (action)
- ajouter l'attribut alt aux images qui n'en ont pas (filtre)

Hook WordPress (ajout alt img)

```
1 function traveloop_add_empty_alt_to_images($content) {
2     if (strpos($content, '<img') === false) {
3         return $content;
4     }
5
6     $content = str_replace(
7         '<img ',
8         '<img alt="" ',
9         $content
10    );
11
12    return $content;
13 }
14 add_filter('the_content', 'traveleop_add_empty_alt_to_images');
```

La sécurité sous WordPress

Il est très important de faire attention à la sécurité sous WordPress.

Les données récupérées via SCF, surtout si celles-ci viennent de contributeurs, peuvent contenir des scripts malveillant (attaques XSS).

Pour éviter cela, il est très important d'échapper les données que l'on récupère. Il existe certaines fonctions permettant de se protéger de ses attaques :

- `esc_html()` → échappe le texte pour éviter le code injecté.
- `esc_attr()` → échappe les valeurs destinées aux attributs HTML.
- `esc_url()` → nettoie et sécurise une URL avant de l'utiliser dans un lien.

La sécurité sous WordPress

Un autre point sensible de WordPress est sa page d'authentification.

Sur tous les sites WordPress, on accède à celle-ci via `/wp-login`.

Certains plugins comme *WPS Hide Login*, permettent de changer cette URL, réduisant drastiquement les possibilités d'attaques par brute-force effectuée par des **crawlers**.

La sécurité sous WordPress

En plus des deux exemples précédents, il existe des automatismes à avoir pour éviter un bon nombre d'attaques :

- 90% des attaques viennent de failles de plugins non mis-à-jour. Il est très important de les mettre à jour aussi régulièrement que possible.
- Ne jamais utiliser d'accès évident (admin + admin).
- Protéger son site via un fichier .htaccess, pour interdire l'accès direct aux fichiers, notamment wp-config.php, qui contient des données sensibles.

Les tâches de cron

Les tâches de cron sont des événements programmés pour s'exécuter à un moment donné ou à intervalles régulières.

Nous pourrions par exemple avoir une tâche de cron qui tourne tous les soirs à 23h pour récupérer des données via une API.

Les tâches de cron fonctionnent avec les fonctions `wp_schedule_event()` et `wp_next_scheduled()`.

Le plugin WP Cronrol propose une interface graphique pour ces tâches.

Theming WordPress Timber



Timber et Twig

Timber est une bibliothèque PHP qui sert de couche intermédiaire pour structurer les données et les passer à Twig afin de rendre les templates plus clairs et maintenables.

Twig est un moteur de templates PHP créé par Symfony qui sert à générer du HTML proprement en séparant la logique métier de la présentation.



Utilisation de Twig et Timber

Pour utiliser Twig et Timber, il est nécessaire d'installer le gestionnaire de paquet 'Composer' (<https://getcomposer.org/>). C'est un outil similaire à npm, mais pour des packages écrits en PHP.

Une fois Composer prêt, il suffit d'installer Timber via la commande Composer :
`composer require timber/timber`

L'ensemble du processus d'installation et la documentation se trouve à l'adresse <https://timber.github.io/docs/v2/>

Pourquoi utiliser Timber et Twig ?

Le code PHP mélangé au HTML dans les fichiers de template rend le code difficile à lire, à maintenir et à tester.

Timber et Twig sépare ce que fais PHP seul :

- Timber (PHP) s'occupe de la logique et des données
- Twig (Moteur de Templates) s'occupe du rendu (HTML)

Twig permet également d'avoir un code plus sécurisé, avec l'échappement automatique des variables par exemple.

Comment WordPress choisit un template

Lorsqu'un utilisateur visite une page, WordPress :

1. Analyse l'URL demandée
2. Identifie le type de contenu
3. Parcourt la template hierarchy
4. Utilise le premier fichier correspondant trouvé (PHP)

Avec Timber, ce fichier PHP :

5. Prépare les données via `Timber::context()`
6. Rend un template Twig avec `Timber::render()`

L'architecture avec Timber

Pour que la séparation entre logique et rendu fonctionne, chaque page nécessite désormais deux fichiers :

Le fichier PHP (Contrôleur) : Il récupère les données via Timber et appelle la vue.

Le fichier TWIG (Vue) : Il reçoit les données et génère le HTML.

```
1 // single.php
2 $context = Timber::context(); // Récupère les données globales (menu, site)
3 $context['post'] = Timber::get_post(); // Récupère le post actuel
4 Timber::render('single.twig', $context); // Affiche la vue
```

Timber::context()

Timber::context() retourne un tableau PHP qui contient les données qui seront envoyées à Twig.

Par défaut, le contexte contient déjà plusieurs informations utiles :

- site → informations du site (nom, url, description...)
- request → informations de la requête en cours
- theme → informations du thème
- user → utilisateur courant (si connecté)

C'est dans ce tableau que nous rajouterons les données que nous voulons lui faire passer.

Syntaxe de base de Twig

```
1  {# Déclarer une variable #}  
2  {% set variable = 'Hello' ~ '...' %}  
3  
4  {# Afficher une variable #}  
5  {{ variable }}  
6  
7  {# Créer un bloc logique #}  
8  {% if variable %}  
9  
10 {# Appliquer un filtre #}  
11 {{ variable|raw }}  
12  
13 {# Appeler une fonction #}  
14 {{ function() }}  
15  
16 {# Parcourir une liste de posts ou un tableau #}  
17 {% for post in posts %}  
18     <h2>{{ post.title }}</h2>  
19 {% else %}  
20     <p>{{ 'No posts found' }}  
21 {% endfor %}
```

Contrairement à PHP qui possède des fonctions pour l'affichage des éléments WordPress (the_title(), the_content(), ...), Twig stocke l'ensemble de ses informations dans un objet post.

L'héritage de template avec Twig

Twig permet aussi d'hériter de certains templates, en remplaçant certaines parties du code.

```
1  {%# base.twig #%}
2  <html>
3    <body>
4      {% include 'header.twig' %}
5      <main>
6        {% block content %}
7          {% endblock %}
8      </main>
9      {% include 'footer.twig' %}
10  </body>
11  </html>
```

```
1  {%# single.twig #%}
2  {% extends "base.twig" %}
3  {% block content %}
4    <h1>{{ post.title }}</h1>
5    <div>{{ post.content }}</div>
6  {% endblock %}
```

Le fichier base.twig

```
1 <!DOCTYPE html>
2 <html {{ site.language_attributes }}>
3 <head>
4     <meta charset="{{ site.charset }}">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>{{ site.name }}</title>
7
8     {{ function('wp_head') }}
9 </head>
10 <body {{ body_class }}>
11
12     {% block header %}
13         {% include 'partials/header.twig' %}
14     {% endblock %}
15
16     <main>
17         {% block content %}{% endblock %}
18     </main>
19
20     {% block footer %}
21         {% include 'partials/footer.twig' %}
22     {% endblock %}
23
24     {{ function('wp_footer') }}
25 </body>
26 </html>
```

Le fichier base.twig sera notre fichier de base.

Les différents blocs pourront être surchargés par nos templates Twig.

Les blocs header et footer fournissent une valeur par défaut dans le cas où ils ne sont pas surchargés.

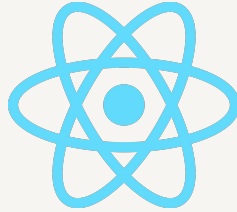
Passage de propriétés aux templates inclus

Lors de l'inclusion d'un template Twig, on peut lui fournir des propriétés.

Celle-ci pourront être utilisées dans le template appelé.

```
1  {% include 'components/country_card.html.twig' with {  
2      title: 'Italie',  
3      user: country_data,  
4      showCard: true  
5  } %}
```

Theming WordPress Headless



Le développement headless

Le développement headless est un paradigme dans lequel le back-end et le front-end sont totalement séparés.

L'un des atouts majeurs du headless est le fait de s'extraire de l'environnement WordPress et d'être libre de choisir la technologie utilisée pour le front-end (React, Angular, ...), WordPress n'agit plus que comme une base de données.

L'autre avantage est le fait que l'on peut faire des sites multi plateformes et des app mobiles, tout en n'ayant qu'une seule source de données.

Les intégrateurs eux, ne changent pas leurs habitudes et travaillent toujours avec le coeur de WordPress.

Rendre SCF accessible - champ unique

SCF n'est pas accessible de base dans l'API REST pour des raisons de confidentialité. Ce code permet de rendre visible un champ dans l'API.

```
1  add_action('rest_api_init', function () {
2      register_rest_field('country', 'country_excerpt', [
3          'get_callback' => function ($post_arr) {
4              return function_exists('get_field')
5                  ? get_field('country_excerpt', $post_arr['id'])
6                  : null;
7          },
8      ]);
9  });
10
```

Rendre SCF accessible - champs multiples

Même code mais pour rendre plusieurs champs disponibles d'un coup.

```
1  add_action('rest_api_init', function () {
2      register_rest_field('country', 'fields', [
3          'get_callback' => function ($post_arr) {
4              if (!function_exists('get_field')) {
5                  return [];
6              }
7
8              return [
9                  'country_excerpt' => get_field('country_excerpt', $post_arr['id']),
10                 'country_price'   => get_field('country_price', $post_arr['id']),
11             ];
12         },
13     ]);
14 });
```


Les APIs WordPress

Par exemple, pour mon site : localhost:10003

<http://localhost:10003/wp-json/wp/v2/country/44>

Me donne toutes les informations sur mon CPT country avec l'ID 44

http://localhost:10003/wp-json/wp/v2/pages?per_page=10

Récupère les 10 dernières pages

<http://localhost:10003/wp-json/wp/v2/posts?slug=hello-world>

Récupère le post avec le slug hello-world

Utilisation de l'API

```
1 import React, { useEffect, useState } from "react";
2 import type { WpCountry } from "../types/wp";
3
4 export default function CountriesList() {
5     const [countries, setCountries] = useState<WpCountry[]>([]);
6
7     useEffect(() => {
8         (async () => {
9             const res: Response = await fetch("http://localhost:10003/wp-json/wp/v2/country?per_page=5");
10             const data: WpCountry[] = await res.json();
11             setCountries(data);
12         })();
13     }, []);
14
15     return (
16         <ul>
17             {countries.map((country) => (
18                 <li key={country.id}>{country.title.rendered}</li>
19             ))}
20         </ul>
21     );
22 }
23
```

CORS

Quand le front et WordPress ne sont pas sur le même domaine/port, le navigateur bloquera les requêtes (CORS error).

Pour permettre au front d'accéder aux données, il faut au choix :

- Autoriser le site à se connecter via les header (souvent depuis le .htaccess) avec Access-Control-Allow-Origin: <https://front.monsite.fr>
- configurer un proxy côté front (Vite / Webpack / Next)

Accès au contenu restreint

Sans authentification, l'API REST renvoie uniquement le contenu publié sur le site.

Pour accéder aux brouillons, aux previews et au contenu privé, il faut une authentification.

On peut utiliser des solutions comme Application Passwords pour permettre cela. Il s'agit de champs à remplir dans la gestion des profils

Your new password for **React headless** is: Copy ✕

Be sure to save this in a safe location. You will not be able to retrieve it.

Name	Created	Last Used	Last IP	Revoke
React headless	January 4, 2026	—	—	<button>Revoke</button>

Name	Created	Last Used	Last IP	Revoke
------	---------	-----------	---------	--------

Accès au contenu restreint

Une fois le mot de passe généré, on peut l'utiliser avec Basic Auth dans notre front.

```
1  useEffect(() => {
2    (async () => {
3      const res: Response = await fetch(
4        "http://localhost:10003/wp-json/wp/v2/posts?status=draft&per_page=5", {
5        headers: {
6          Authorization: "Basic " + btoa("admin:APP_PASSWORD"),
7        },
8      }
9    );
10
11    const data: WpCountry = await res.json();
12    setCountries(data);
13  })();
14 }, []);
```

Mise en garde sur le headless

Le headless est un outil extrêmement puissant, mais c'est un choix d'architecture lourd.

En plus du risque de fuite de données, vous devez gérer :

- 2 stacks (back + front) et leurs déploiements
- SEO/SSR/preview
- fonctionnalités WP à reconstruire

Règle simple : **si vous n'avez pas une raison forte, ne faites pas headless.**